Classical AI Reasoning Agent

Natural-language knowledge base → internal representation → classical reasoning → True / False

Name: Deep Das Student ID: M26MAC004

1. Overview

The system decides whether a statement is entailed by a knowledge base written in English. Each example goes through three stages: a rule-based semantic parser builds a symbolic representation, a router picks the engine that suits it, and that engine computes the answer by classical inference. No machine-learning model or generative AI is used at any stage. The engine is plain Python with only the standard library; pandas and datasets are used for loading data.

Routing order: (1) causal language (“A causes B … Would Y occur if not X instead of X?”) goes to the counterfactual engine; (2) facts and rules go to forward chaining, parsed either as first-order rules with variables (“If someone is young then they are smart”) or as ground rules with fixed connectives (“Assuming the river is fast, we can say the house volunteers”), whichever covers more sentences; (3) loosely paraphrased rules use a lexicon-based extractor plus forward chaining; (4) any other English goes through a shallow NL→FOL parser to the resolution prover. If a test file also carries first-order formulas, those are sent straight to the prover. If nothing can be proved, the answer is False.

2. Representation

• Literals and Horn rules. A fact is a signed atom, e.g. young(bob), ¬chase(cow, cat). Adjectives become unary predicates and transitive verbs binary predicates (lemmatised: “chases” → chase). Generic subjects (someone, something, they, people, things) become a variable ?x, so “All red, cold things are round” becomes red(?x) ∧ cold(?x) → round(?x).

• Ground rules. In jise/ruletaker every clause is about a named entity and rules use five fixed connectives (If…then, Assuming…we can say, Based on…it is evident that, Given that…it follows that, When…we can conclude). Each clause becomes one propositional atom, normalised by lower-casing, dropping articles and lemmatising (“Sarah carries bags” and “sarah carries bags” both give [sarah carry bag]).

• World assumption. RuleTaker is closed-world: a statement that cannot be derived is False, and ¬p holds when p is not derivable (negation as failure). An open-world mode with classical negation is also implemented.

• Structural causal model. Each CounterBench clause becomes a Boolean equation: “A causes not B” → B := ¬A; “A and B together cause C” → C := A ∧ B; “A or not B causes C” → C := A ∨ ¬B. Variables without an equation are exogenous; “X~Bern(p)” gives one a prior, “We observed X” is evidence, and the question supplies the intervention do(X = 0).

• First-order logic. Formulas (∀ ∃ ¬ ∧ ∨ → ↔ ⊕ =) are parsed into a syntax tree, put into negation normal form, Skolemised and distributed into clauses over terms with variables and Skolem functions.

3. Reasoning algorithms

• Unification (Robinson, with occurs check) is shared by all engines.

• Forward chaining with stratified negation as failure. Predicates get strata from the rule dependency graph, so a negated predicate is complete before it is tested. Each stratum is run to a fixpoint; rule bodies are matched against an index of facts, positive literals first. Every derived fact records its rule and premises, which yields the proof trace.

• Backward chaining (SLD-style goal reduction) is implemented as an independent check and agrees with forward chaining.

• Counterfactuals by search over possible worlds (abduction → action → prediction). A depth-first enumeration of the exogenous variables keeps the worlds consistent with what actually happened; in each, the intervention overrides X and the equations are propagated forward. The answer is yes when the prior-weighted share of consistent worlds where the outcome holds exceeds 0.5.

• Resolution refutation. KB ⊨ φ iff KB ∪ {¬φ} is unsatisfiable. The prover uses set of support (only clauses descending from ¬φ are expanded), always expands the lightest clause next (uniform-cost search on clause size), and applies binary resolution, factoring, tautology deletion, duplicate and subsumption checks, a term-depth bound, and a 0.4 s / 250-clause budget per query.

4. Experiments on the public data

Dataset	Queries	Accuracy %	Majority %	Time (s)	ms / q
RuleTaker	1625	98.46	54.22	0.84	0.52
CounterBench	1200	75.00	50.42	0.35	0.29
FOLIO (from English)	1204	62.21	61.79	0.48	0.40
All public queries, end-to-end	**4029**	**80.64**		**1.67**	**0.41**
FOLIO given gold FOL (engine check)	1204	81.06	61.79	21.43	17.80

End-to-end means the agent sees only the English context and statement. The last row gives the resolution prover FOLIO's hand-written first-order formulas instead of the English; it measures the reasoning engine alone and is not counted in the total. Labels are binary (True = entailed): FOLIO "False" and "Uncertain" both count as False, CounterBench "no" as False. "Majority" is the accuracy of always answering the more frequent label. Times include parsing.

Observations:

• RuleTaker. The ground-rule parser recognises all five connective templates, strips the "Facts:" / "Rules:" headers and normalises capitalisation, so every sentence of a jise/ruletaker context is parsed and the answer follows exactly from forward chaining under the closed world.

• FOLIO from English is the weak point. Its sentences are long and varied ("People in this club either perform in school talent shows often or are inactive…"). The shallow NL→FOL parser rarely produces matching predicates, so accuracy stays close to the majority baseline. Translating free English into logic without a learned model is the main open problem in this design.

• FOLIO with its formulas. Given the gold formulas, the prover solves most stories. Nearly all remaining misses come from slips in the annotations themselves (misspelt constants such as "bonne" for "bonnie", predicates named differently in premise and conclusion), which no sound prover can bridge. The parser tolerates unbalanced brackets in these formulas.

• Prover budget. Lowering the per-query budget from 2.0 s / 600 clauses to 0.4 s / 250 gave the same FOLIO accuracy at about a quarter of the time.

• Routing safeguards. The ground-rule parser rejects clauses containing words that need variables (someone, people, all, either …), and the paraphrase extractor only runs on short sentences. Both prevent false proofs on FOLIO stories that look superficially like RuleTaker text.

5. Reasoning traces

Assignment example (forward chaining with variables) — predicted True

Context: Bob is young. Bob is kind. If someone is young, then they are smart. If someone is smart and kind, then they are successful.

Statement: Bob is successful.

```
engine: forward chaining (CWA)
facts=4 rules=2 firings=5
query proved:
successful(bob)   [rule 1: smart(?x) ∧ kind(?x) → successful(?x)]
  smart(bob)   [rule 0: young(?x) → smart(?x)]
    young(bob)   [fact]
  kind(bob)   [fact]
```

RuleTaker (gold: True) — predicted True

Context: Facts: The globe is colorful. The globe is friendly. The globe is colorful. Rules: Given that the globe is colorful, it follows that mike is colorful. If the globe is friendly, then the dog runs marathons. Given that the city writes stories, it follows that mike is colorful. Given that the dog runs …

Statement: Mike is colorful.

```
engine: forward chaining over ground rules (CWA)
facts=6 rules=6 firings=13
query proved:
[mike be colorful]   [rule 0: [globe be colorful] → [mike be colorful]]
  [globe be colorful]   [fact]
```

RuleTaker (gold: False) — predicted False

Context: Facts: The wall dances gracefully. The river dances gracefully. The river becomes famous. Rules: Assuming the river dances gracefully, we can say the river is fast. If the wall dances gracefully, then the wall dances gracefully. Based on the river becomes famous, it is evident that the wall dances g …

Statement: The sandwich dances gracefully.

```
engine: forward chaining over ground rules (CWA)
facts=4 rules=6 firings=9
query [sandwich dance gracefully] is not derivable from the KB -> False
```

CounterBench (gold: True) — predicted True

Context: We know that Ziklo causes Blaf, Blaf and Ziklo together cause Trune, Trune causes not Vork, and Vork causes Lumbo.

Statement: Would Lumbo occur if not Ziklo instead of Ziklo?

```
engine: abduction-action-prediction over possible worlds
blaf := ziklo
trune := (blaf ∧ ziklo)
vork := ¬trune
lumbo := vork
roots=['ziklo']
evidence={'ziklo': True}
do={'ziklo': False}
consistent worlds=1
P(lumbo | evidence, do)=1.000
```

FOLIO, prover given the first-order annotations (gold: True) — predicted True

Context: All people who regularly drink coffee are dependent on caffeine. People regularly drink coffee, or they don't want to be addicted to caffeine, or both. No one who doesn't want to be addicted to caffeine is unaware that caffeine is a drug. Rina is either a student who is unaware that caffeine is a dr …

Statement: Rina doesn't want to be addicted to caffeine or is unaware that caffeine is a drug.

```
engine: resolution refutation (FOL)
⊥ (empty clause: contradiction)   [res]
  IsDependentOn(rina, caffeine)   [res]
    DrinkRegularly(rina, coffee)   [res]
      WantToBeAddictedTo(rina, caffeine)   [goal]
      ¬WantToBeAddictedTo(?x8269, caffeine) ∨ DrinkRegularly(?x8269, coffee)   [kb]
    IsDependentOn(?x8268, caffeine) ∨ ¬DrinkRegularly(?x8268, coffee)   [kb]
  ¬IsDependentOn(rina, caffeine)   [res]
    ¬Student(rina)   [res]
      AwareThatDrug(rina, caffeine)   [goal]
      ¬Student(rina) ∨ ¬AwareThatDrug(rina, caffeine)   [kb]
    Student(rina) ∨ ¬IsDependentOn(rina, caffeine)   [kb]
```

6. Limitations

• FOLIO from English: accuracy near the majority baseline. A stronger semantic parser (for instance a model that only translates English into formulas, with this prover still making the decision) would be the next step.

• The rule parsers cover the sentence patterns found in RuleTaker (attributes, relations, negation, conjunctions, If/All rules, the five connective templates); unfamiliar phrasings fall back to approximate lexicon-based extraction.

• Counterfactual answers rely on the CounterBench templates (causes / causes not / together cause / or / Bern(p) / We observed / Assume … further suppose).